

Replit ↔ Canonical Sync Guide

Purpose: bring the **Replit** copy of Mantua.AI into an *exact match* with the canonical repository. Follow this top-to-bottom. The fastest, most reliable path (Section 1) makes the code byte-identical to `main`; Sections 2–6 let you verify or hand-apply if a full git sync isn't possible.

- **Canonical repo:** `https://github.com/DelleonMcglone/Mantua-Intelligence`
 - **Authoritative branch:** `main`
 - **Reference commit at time of writing:** `e010c6d`
 - **Runtime:** Node **22.x**, npm workspaces (`client` , `server`)
-

1. Fastest path — make Replit identical to `main` (recommended)

This is the "exact match." Run it in the Replit shell.

⚠ This **overwrites local Replit changes** to tracked files. If the Replit copy has unique work you want to keep, do Section 1b instead.

```
# From the repo root in Replit:
git remote -v                # confirm/inspect existing remote
# If no 'canonical' remote yet:
git remote add canonical https://github.com/DelleonMcglone/Mantua-Intelligence.git
git fetch canonical
git checkout main 2>/dev/null || git checkout -b main
git reset --hard canonical/main # <-- exact match to canonical main
npm install                  # resync dependencies (adds serverless-http, etc.)
```

1b. Keep Replit-only changes, then align

```
git remote add canonical https://github.com/DelleonMcglone/Mantua-Intelligence.git
git fetch canonical
git stash                # park local edits
git merge canonical/main # bring canonical in; resolve conflicts toward canonical
git stash pop            # reapply your edits on top (resolve as needed)
npm install
```

After either path, jump to **Section 5 (Environment)** and **Section 6 (Verify)**.

2. Architecture (what the canonical version is)

Monorepo, two npm workspaces plus a serverless endpoint:

```
client/    Vite + React + TypeScript frontend (port 5173 in dev)
server/    Express + TypeScript API, run via tsx (port 3001 in dev)
api/       Vercel serverless endpoint (wraps the Express app)
contracts/ Foundry contracts (Stable Protection hook lives in a submodule)
vercel.json Deploy config (client build + serverless API)
```

- **Network:** Base Sepolia testnet (chainId 84532). MANTUA_NETWORK=testnet.
- **Auth:** Privy. **DB:** Postgres (via Drizzle). **Chain reads:** viem.
- The server is **run by tsx / bundled by Vercel — never compiled with tsc** (source uses explicit `.ts` import extensions).

3. Required structure for the serverless/deploy setup

If the Replit copy predates the deploy work, these files **must exist** and match:

- **server/src/app.ts** — Express app *factory*. Builds `app`, registers all middleware + routers, exports `app`. **Does not** call `.listen()`.
- **server/src/index.ts** — local listener only: `ts import { app } from './app.ts'; import { env } from './env.ts'; import { logger } from './lib/logger.ts'; app.listen(env.PORT, () => { logger.info({ port: env.PORT, env: env.NODE_ENV }, 'server listening'); });`
- **api/index.ts** — serverless endpoint: `ts import serverless from 'serverless-http'; import { app } from '../server/src/app.ts'; export default serverless(app);`
- **server/package.json** — has dependency `"serverless-http": "^4.0.0"`.
- **vercel.json** (repo root): `json { "$schema": "https://openapi.vercel.sh/vercel.json", "buildCommand": "npm run build -w @mantua/client", "outputDirectory": "client/dist", "installCommand": "npm install --no-package-lock", "framework": null, "functions": { "api/index.ts": { "memory": 1024, "maxDuration": 30 } }, "rewrites": [{ "source": "/api/(.*)", "destination": "/api/index" }, { "source": "/(.*)", "destination": "/index.html" }] }`
- **eslint.config.js** — the out-of-project override includes `api/**/*.ts`: `js files: ["api/**/*.ts", "contracts/script/**/*.ts", "eslint.config.js", "*.config.{js,cjs,mjs,ts}"]`,

`installCommand: --no-package-lock` is **required** for Vercel: the macOS lockfile otherwise makes npm skip the Linux `@rolldown/binding-linux-x64-gnu` native binary and `vite build` fails (npm optional-deps bug npm/cli#4828).

4. Feature changes to verify (apply if Replit diverged)

These are the behavior changes the canonical version carries. Each lists the file and the exact expected state.

4.1 Landing-page faucet FAQ — `client/src/components/landing/LandingPage.tsx`

FAQ "How can I provide liquidity?" lists **exactly two** faucets: - **Coinbase CDP Faucet** — ETH, USDC, EURC and cbBTC - **Circle Faucet** — USDC, EURC and cirBTC

The **Optimism Faucet** — ETH entry is **removed**.

4.2 Connected-wallet menu — `client/src/components/shell/WalletMenu.tsx`

Below "Get testnet ETH (CDP)" there is a second link **"Get testnet USDC (ARC)"** → `https://faucet.circle.com/` (const `ARC_FAUCET_URL`).

4.3 Analyze topics — `server/src/lib/analyze.ts` + `client (AnalyzePanel.tsx, lib/chat-intent.ts, App.tsx)` + tests

- Topic `usdc-usdt-pool` → `usdc-eurc-pool` (DefiLlama filter `USDC/EURC`, titles/summaries say `USDC/EURC`).

- Topic `top-rwa-tokens` → `top-stablecoins` (token list: USDC, USDT, DAI, EURC, agEUR; ranked by 24h change; labels show (USD peg) / (EUR peg)).
- `mantuaHooks()` lists **Stable Protection first, Dynamic Fee second** (summary and bullets).
- Suggestion chips read **"Analyze USDC/EURC pool health"** and **"Show me top performing Stablecoins"**.
- Intent rules: `stablecoins` matches `\bstablecoins?\b`; `USDC/EURC pool` matches `\busdc\b && \beurc\b`. Update `AnalyzeTopic` unions in `App.tsx`, `lib/chat-intent.ts`, and `TOPICS` in `analyze.ts` + the two test files.

4.4 API base URL — `client/.env.example` and `client/.env.local`

`VITE_API_BASE_URL` is **empty** by default (use the Vite dev proxy; the dev server has **no CORS** middleware — a cross-origin value causes "failed to fetch").

4.5 Create Pool layout — `client/src/features/liquidity/AddLiquidityForm.tsx`

The token-input grid uses `gridTemplateColumns: "minmax(0, 1fr) auto minmax(0, 1fr)"` (not `1fr auto 1fr`) so the cards shrink and the form fits with no horizontal scroll.

4.6 Swap panel feedback — `client/src/features/swap/TestnetSwapPanel.tsx`

- The "Swap rejected by hook" / "no pool" message is gated on `amountEntered` (does **not** render on mount).
- A successful quote returning `0` out → flag `noLiquidity`; shows **"Insufficient liquidity ..."**, relabels the CTA, and disables the swap.

4.7 Quote-failure reason — `server/src/lib/v4-onchain-swap.ts`

In `findMaxQuotableInputV4`, when a revert can't be decoded, it reads `slot0` (`readSlot0`) to distinguish an **initialized** pool ("hook paused / liquidity out of range") from a **missing** pool — instead of the generic "missing liquidity".

4.8 Stable Protection 1:1 peg init (important behavior)

- `server/src/lib/sqrt-price.ts` exports `SQRT_PRICE_X96_1_1` ($= 2^n \cdot 96^n$).
- `server/src/routes/pool-create.ts`: when `hook === "stable-protection"`, the pool is initialized at `SQRT_PRICE_X96_1_1` (price 1.0), not the amount-derived market price. Other hooks/no-hook keep the market price.
- `client/src/features/liquidity/AddLiquidityForm.tsx`: `priceRatioAtoB` returns `1` when `hook === "stable-protection"` so entered amounts mirror 1:1.

Rationale: the Stable Protection hook models USDC/EURC as a **1:1 peg**. Initializing off-peg lands the pool in the circuit-breaker zone and blocks all swaps (`CircuitBreakerTripped`). Pre-existing off-peg pools stay bricked at their fee tier — create at a different fee tier for a fresh 1:1 pool.

5. Environment & local run

Server env (`server/.env`, git-ignored) — required to boot

```
DATABASE_URL=postgres://<user>@<host>:5432/<db> # hosted Postgres (e.g. Neon) for Replit
PRIVY_APP_ID=cmofvi87700dq0ci88olare5n
PRIVY_APP_SECRET=<your privy app secret>
MANTUA_NETWORK=testnet
# optional: BASE_RPC_URL, CDP_*, ANTHROPIC_API_KEY, THE_GRAPH_API_KEY
```

Client env (`client/.env.local` , git-ignored)

```
VITE_PRIVY_APP_ID=cmofvi8770dq0ci88olare5n
VITE_API_BASE_URL= # EMPTY in dev (proxy). Only set to the API URL in prod.
VITE_MANTUA_NETWORK=testnet
```

Database

Replit has no local Postgres — use a hosted one (Neon free tier works). Then:

```
npm run db:migrate -w @mantua/server
```

Run (two processes)

```
npm install
npm run dev:server # http://localhost:3001
npm run dev:client # http://localhost:5173
```

On Replit, expose port **5173** (the client). The client proxies `/api` → `3001`.

6. Verify the match

```
# Code is identical to canonical main:
git fetch canonical && git diff --stat canonical/main # expect: no output

# Client builds (the Vercel command):
npm run build -w @mantua/client # expect: success

# Quick behavioral checks in the running app:
# - Swap card opens with NO error before entering an amount
# - Create Pool form fits with no horizontal scroll
# - Analyze chips read "USDC/EURC pool health" + "top performing Stablecoins"
# - Wallet menu shows "Get testnet USDC (ARC)"
```

Note: `npm run typecheck` currently fails on the **server** workspace (pre-existing `tsconfig baseUrl` deprecation masking a backlog of server type errors). This is the same on canonical `main` and does **not** affect running the app, the client build, or the Vercel deploy. Don't treat it as a sync failure.

TL;DR for the Replit agent

1. Add `canonical` remote → `git fetch canonical` → `git reset --hard canonical/main` → `npm install`.
2. Ensure `api/index.ts`, `server/src/app.ts`, slim `server/src/index.ts`, `vercel.json`, and `serverless-http` exist (Section 3).
3. Set env vars (Section 5); point `DATABASE_URL` at a hosted Postgres and run migrations.
4. Keep `VITE_API_BASE_URL` empty in dev. Expose port 5173.
5. Verify with `git diff --stat canonical/main` (should be empty).